Universidad Autónoma del Estado de Hidalgo

Escuela Superior de Tlahuelilpan

REPORTE DE PRÁCTICA No. 8

PROGRAMACIÓN CON PYTHON

Yahir Alexander Orta Rivero

Mónica Cornejo Velázquez

Grupo: 301

Introducción.

Python es un lenguaje de programación de propósito general muy poderoso y flexible, a la vez que sencillo y fácil de aprender. Este lenguaje fue creado a principios de los noventa por Guido van Rossum en los Países Bajos. Es software libre, y está implementado en todas las plataformas y sistemas operativos habituales.

Toma características de lenguajes predecesores, incluso, compatibilizando la solución de varios de ellos. Por ejemplo, habilita tres formas de imprimir el valor de una variable: desde el entorno interactivo escribiendo su nombre (como en Basic), usando la función print, con concatenación de elementos (al estilo del write de Pascal) o bien con patrones de formato (al estilo del printf de C).

Python se desarrolla bajo una licencia de Open source o código abierto aprobada por OSI, por lo que se puede usar y distribuir libremente, incluso para uso comercial. La Python Software Foundation (PSF) es una corporación sin fines de lucro 501 (c) (3) que posee los derechos de propiedad intelectual detrás del lenguaje de programación Python. Administramos las licencias de código abierto para Python versión 2.1 y posteriores.

El Python Package Index (PyPI) o en español significa Índice de paquetes de Python alberga miles de módulos de terceros para Python.

Tanto la biblioteca estándar de Python como los módulos aportados por la comunidad permiten infinitas posibilidades.

- Desarrollo web e Internet.
- Acceso a la base de datos.
- GUIs de escritorio.
- Científico y numérico.
- Educación.
- Programación de red.
- Desarrollo de Software y Juegos.

Las características del lenguaje de programación Python se resumen a continuación:

- Es un lenguaje interpretado, no compilado, usa tipado dinámico, fuertemente tipado.
- Es multiplataforma, lo cual es ventajoso para hacer ejecutable su código fuente entre varios sistemas operativos.

- Es un lenguaje de programación multiparadigma, el cual soporta varios paradigma de programación como orientación a objetos, estructurada, programación imperativa y, en menor medida, programación funcional.
- En Python, el formato del código (p. ej., la indentación) es estructural.

En Python tiene varios tipos de datos compuestos estándar disponibles por defecto en el intérprete, como los tipos numéricos, secuencias, mapeos y conjuntos usados para agrupar otros valores.

Para el caso de las estructuras de datos se usan variables y constantes las cuales usan operadores para tratar los tipos de datos estándar.

Objetivo General.

Desarrollar una interfaz visual en Python que es permita hacer uso de diversos factores de este paradigma de programación.

Objetivos Específicos.

- Programar un sistema similar a un punto de venta sin bases de datos
- Generar las interfaces necesarias que permitan la manipulación de la información de entrada, dentro del sistema y de salida del sistema
- Crear interfaces claras, intuitivas y atractivas para los usuarios.

Desarrollo de la actividad práctica

Mi proyecto en Python

Desarrollar en Python, las interfaces necesarias para realizar un sistema de nombre "tienda" que se relacione con la actividad de una pequeña empresa que vende artículos a clientes.

- La empresa debe almacenar información de cada uno de sus clientes.
- La empresa debe almacenar información de los artículos que vende.
- El negocio debe almacenar datos de las ventas que realizan.

Toda venta genera un ticket o nota de venta con su correspondiente detalle que describe:

- Producto (s) que se vende
- Cantidad
- El precio unitario del producto
- Importe del producto (Cantidad * Precio)

(Similares a los que te dan impresos en las tiendas, solo que este será solo digital)

Al final del ticket incluir:

Subtotal = Sumatoria del importe del producto

Iva = Subtotal * .16

Total = Subtotal + iva

El sistema debe incluir un inicio de sesión con la identificación de los diferentes roles necesarios para la actividad.

Sistema desarrollado

Para cumplir con la actividad se desarrolló un sistema de punto de venta llamado **"Tienda"**, utilizando Python y la librería tkinter. El sistema administra clientes, artículos y ventas, y genera un ticket digital con el detalle de cada venta, el subtotal, el IVA y el total. A continuación se describe cómo se construyó.

Herramientas utilizadas

Herramienta	Uso dentro del sistema
Python 3	Lenguaje de programación en el que se escribió todo el sistema.
tkinter	Creación de las ventanas y los widgets: `Tk`, `Frame`, `Label`, `Entry`, `Button`, `Text`, `Scrollbar`, `Listbox` y `StringVar`. Se importó con `from tkinter import *`.
tkinter.ttk	`Notebook` para organizar el sistema en pestañas, `Combobox` para elegir al cliente y al producto, y `Treeview` para las tablas de clientes y artículos.
tkinter.messagebox	Ventanas emergentes de información, advertencia y error.
datetime	Obtener la fecha y la hora que se guardan en cada venta (`datetime.now().strftime()`).
img.ico	Archivo de ícono que se asigna a la ventana de inicio de sesión con `iconbitmap()`.

Almacenamiento de la información

Como el sistema no utiliza bases de datos, la información se genera desde el programa y se guarda en variables. Se usaron listas de diccionarios: cada diccionario representa un registro (un cliente, un artículo o una venta) y la lista funciona como una tabla. Los usuarios se guardan en un diccionario cuya clave es el nombre de usuario.

- `usuarios`: diccionario con la contraseña y el rol de cada usuario.
- `clientes`: lista de diccionarios con `id`, `nombre` y `telefono`.
- `articulos`: lista de diccionarios con `id`, `nombre`, `precio` y `cantidad` (existencia).
- `ventas`: lista de diccionarios con `folio`, `fecha`, `cliente`, `vendedor`, `detalle`, `subtotal`, `iva` y `total`.
- `carrito`: lista temporal, creada dentro de `tienda()`, con los productos de la venta en curso.

Los datos con los que inicia el sistema son los siguientes:

Clientes

ID	Nombre	Teléfono
C1	Público en General	-
C2	Yahir Alexander Orta	7721795916
C3	Melanie Bautista	7731569874
C4	Drake Jose Reyes	774569789
C5	Ruth Valeria Soto	7731235697

Artículos

ID	Nombre	Precio	Existencia
P1	Laptop	$15,250.00	16
P2	Lego	$1,100.00	15
P3	Converse	$950.00	16
P4	Carro rc	$520.00	16
P5	Refrigerador	$8,500.00	15
P6	Estufa	$6,950.00	16

Como no se usa un archivo ni una base de datos, los cambios se conservan mientras el programa está abierto (incluso si se cierra sesión y se vuelve a entrar) y se pierden al cerrar el programa por completo.

Roles y permisos

El sistema cuenta con dos roles. Al iniciar sesión, la función `login()` valida el usuario y la contraseña contra el diccionario `usuarios` y devuelve el usuario y su rol; con ellos se abre la ventana principal con `tienda(*sesion)`. Según el rol se agregan o no las pestañas del sistema (`if rol == "Administrador"`).

Rol	Usuario / contraseña	Permisos
Administrador	admin / 123	Ventas, Clientes, Artículos (agregar, editar y eliminar) e Historial de ventas.
Vendedor	vendedor / 123	Ventas y Clientes (puede registrar clientes). No tiene acceso a Artículos ni al Historial de ventas.

Cada venta guarda en el campo `vendedor` el nombre del usuario que la realizó.

Diseño de la interfaz

La interfaz se construyó con ventanas de tkinter; los elementos se acomodan con `pack` y `grid` y las secciones se agrupan en marcos (`Frame`). El sistema tiene las siguientes pantallas:

- **Inicio de sesión:** ventana de 420 × 320 píxeles, que no cambia de tamaño y aparece centrada en la pantalla. Tiene un marco de color verde claro con los campos Usuario y Contraseña (esta se oculta con asteriscos, `show="*"`) y el botón “Iniciar sesión”. Al entrar muestra un mensaje de bienvenida con el usuario y el rol; si los datos son incorrectos muestra un error y limpia la contraseña.
- **Ventana principal:** el título muestra el usuario y el rol (“Tienda - usuario (rol)”). Incluye el botón “Cerrar sesión” y las pestañas del sistema (`ttk.Notebook`).
- **Pestaña Ventas:** a la izquierda están las listas desplegables de cliente y producto, la cantidad y los botones “Agregar”, “Cancelar venta” y “Generar venta y ticket”; a la derecha, un cuadro de texto con barra de desplazamiento donde se muestra el ticket en tiempo real.
- **Pestaña Clientes:** tabla (`Treeview`) con ID, nombre y teléfono, y campos para registrar un cliente nuevo.

- **Pestaña Artículos (solo administrador):** tabla con ID, nombre, precio y existencia, campos de captura y los botones “Agregar / Editar” y “Eliminar seleccionado”. Al seleccionar un artículo sus datos se cargan en los campos.
- **Pestaña Historial de ventas (solo administrador):** lista con el folio, la fecha, el cliente, el vendedor y el total de cada venta.

Para que las columnas del ticket queden alineadas se usó la fuente monoespaciada Courier y cadenas con formato (f-strings) que fijan el ancho de cada columna, por ejemplo `{p['precio']:>10.2f}`. El ticket tiene un ancho de 42 caracteres.

Funciones principales del programa

Función	Descripción
`login()`	Crea la ventana de inicio de sesión y devuelve una lista con el usuario y el rol (o una lista vacía si se cierra la ventana).
`entrar()`	Función interna de `login()`: valida usuario y contraseña, muestra el mensaje de bienvenida o de error y cierra la ventana si el acceso es correcto.
`tienda(usuario, rol)`	Crea la ventana principal con sus pestañas y aplica los permisos según el rol.
`mostrar_ticket(encabezado)`	Calcula el total, el subtotal y el IVA de la venta en curso, redibuja el ticket y devuelve `(subtotal, iva)`.
`agregar()`	Valida el producto, que la cantidad sea entera y mayor que cero y que haya existencia; agrega el producto al carrito.
`cancelar()`	Vacía el carrito y deja el ticket en espera de productos.
`generar_venta()`	Valida cliente y productos, calcula folio y fecha, descuenta existencias, guarda la venta, limpia la pantalla y muestra la confirmación.
`agregar_cliente()`	Valida que ID y nombre no estén vacíos y registra al cliente en la lista `clientes`.
`seleccionar_art(event)`	Al seleccionar un artículo de la tabla, copia sus datos a los campos de captura.
`guardar_articulo()`	Alta o edición de artículos: si el ID ya existe actualiza el registro y si no, lo agrega.
`eliminar_articulo()`	Elimina de la lista el artículo seleccionado en la tabla.
`refrescar()`	Vuelve a llenar las listas desplegables, las tablas de clientes y artículos y el historial cada vez que cambian los datos.
Bloque principal	`while True` que muestra el inicio de sesión y abre la tienda; al cerrar sesión regresa al login y si se cierra el login termina el programa.

Proceso de venta y cálculo del ticket

Una venta se realiza con los siguientes pasos:

1. El vendedor elige al cliente en la lista desplegable (por defecto aparece “C1 - Público en General”).
2. Elige el producto en la lista desplegable y escribe la cantidad. Con el botón “Agregar” el sistema valida que la cantidad sea un número entero mayor que cero y que haya existencia suficiente, tomando en cuenta lo que ya está en la venta.
3. El sistema calcula el importe de cada renglón (cantidad × precio unitario) y actualiza el ticket en pantalla. Si el vendedor se equivoca puede usar “Cancelar venta”, que vacía la lista de productos.
4. Con cada producto agregado se recalculan el Subtotal, el IVA (16 %) y el Total.
5. Con “Generar venta y ticket” el sistema valida que haya cliente y productos, asigna un folio consecutivo y la fecha, descuenta las existencias del inventario, guarda la venta en la lista `ventas`, limpia la pantalla y muestra un mensaje de confirmación.

Como los precios de los artículos ya incluyen el IVA, el sistema obtiene el subtotal dividiendo el total entre 1.16 y el IVA como la diferencia; así se cumple que IVA = Subtotal × 0.16 y Total = Subtotal + IVA. Las fórmulas usadas son:

total = suma de los importes (cantidad * precio)
subtotal = total / 1.16
iva = total - subtotal

Ejemplo del ticket que muestra el sistema al vender 1 Laptop y 2 Lego (mientras la venta está en curso el encabezado dice “VENTA EN CURSO”):

VENTA EN CURSO
==========
Producto Cant P.Unit Importe

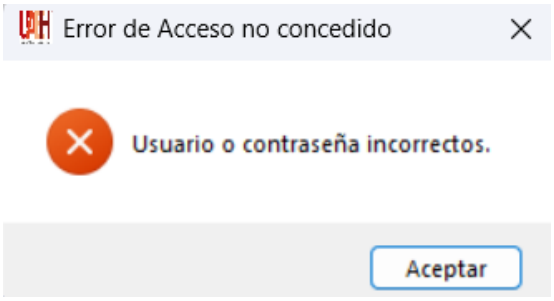
Laptop 1 15250.00 15250.00
Lego 2 1100.00 2200.00

Subtotal: 15043.10
IVA (16%): 2406.90
TOTAL: 17450.00

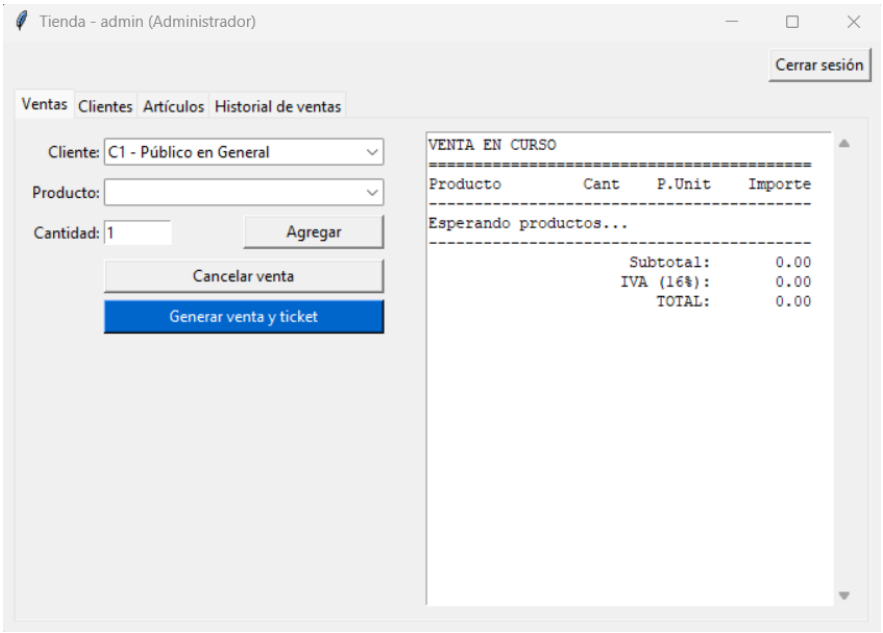
Resultados

Inicio de Sesión. Si el usuario o la contraseña son incorrectos, el sistema muestra un mensaje de error y no permite el acceso.

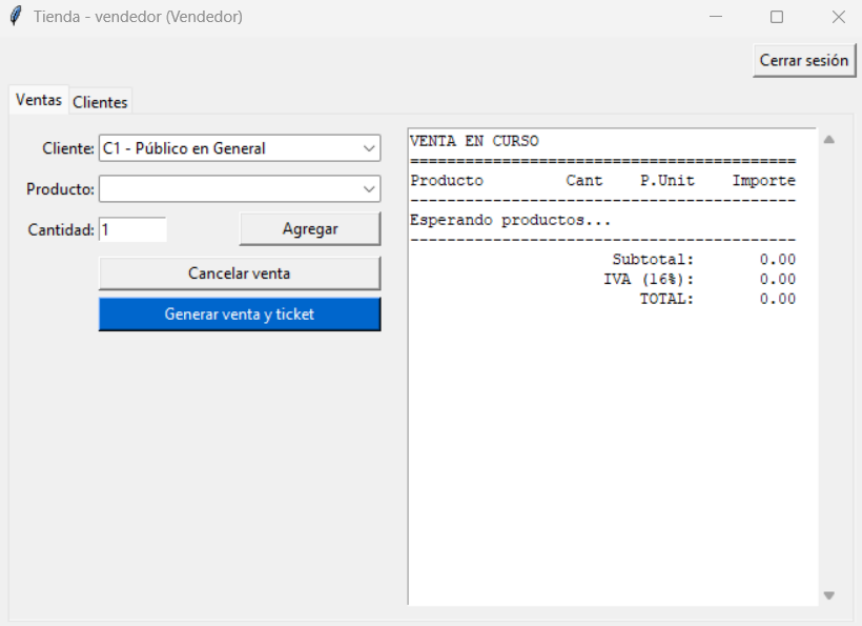




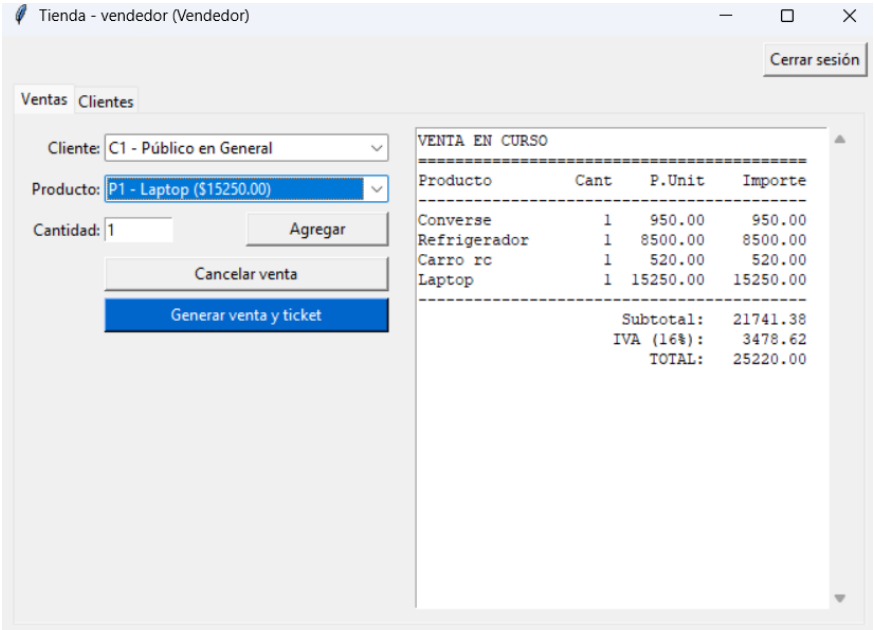
Sesión de admin. Al entrar se muestra la ventana principal con las cuatro pestañas: Ventas, Clientes, Artículos e Historial de ventas.

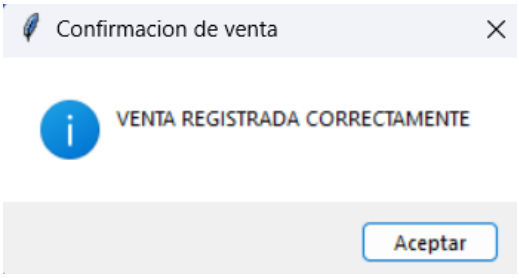


Sesión vendedor. Solo se muestran las pestañas Ventas y Clientes.

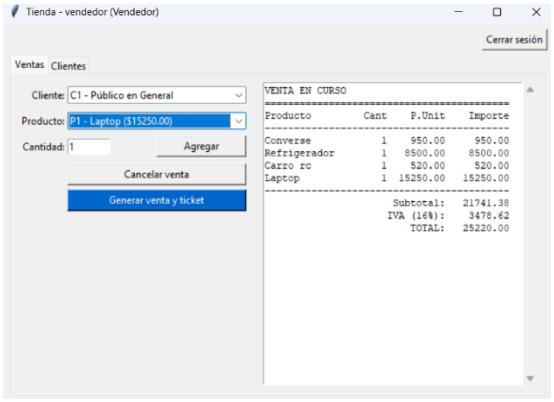


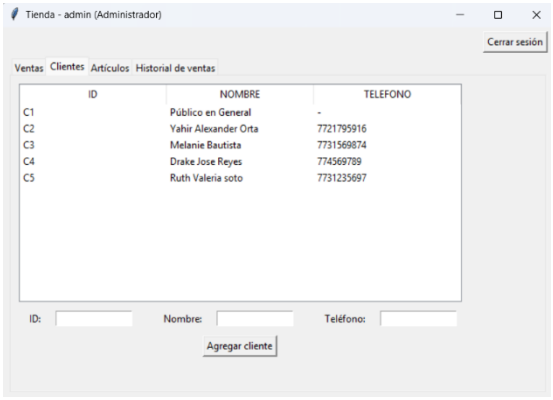
Registro de venta. Se eligen el cliente y los productos, el ticket se va armando en pantalla con el subtotal, el IVA y el total, y al generar la venta el sistema muestra un mensaje de confirmación.

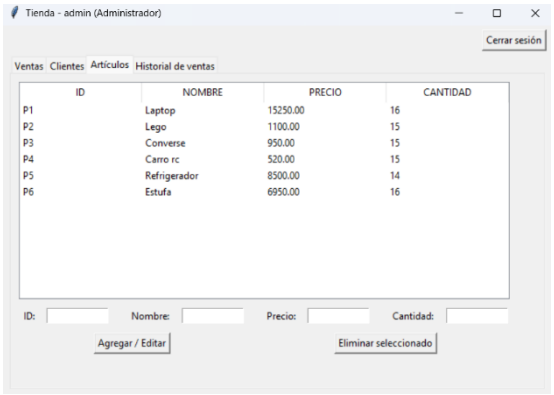


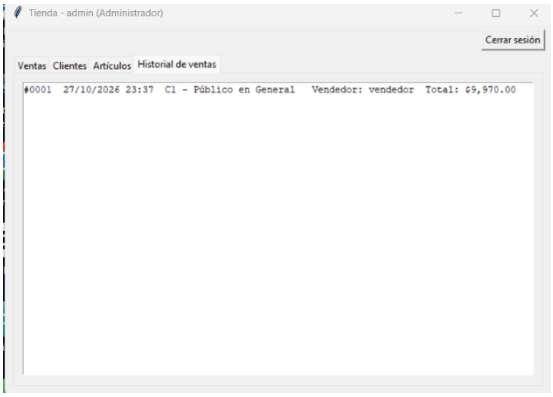


Navegación

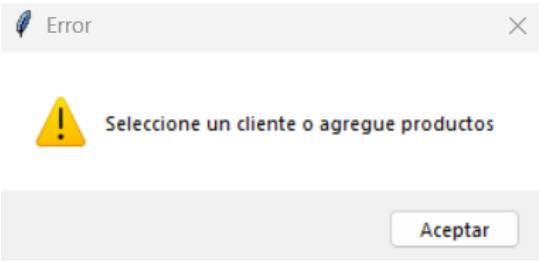




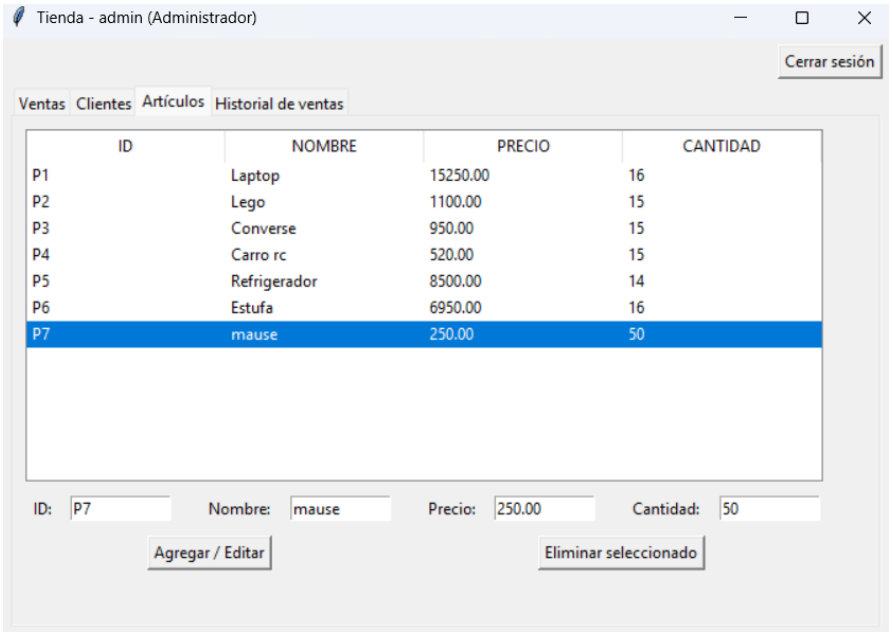




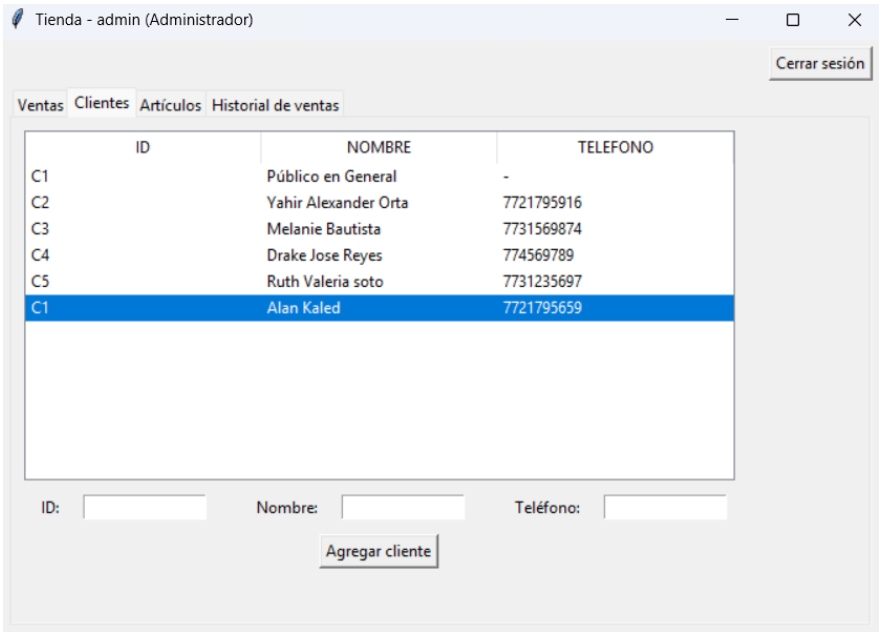
Inventario insuficiente. Si la cantidad es mayor que la existencia, el sistema muestra un aviso y no agrega el producto.



Agregar artículo. Solo el administrador puede agregar, editar y eliminar artículos.



Agregar clientes. Tanto el administrador como el vendedor pueden registrar clientes.



Discusión

La programación orientada a eventos fue la base del sistema. En un programa de consola, el programador decide el orden en que se pide la información, se procesa y se muestra el resultado. En una interfaz gráfica ese orden lo decide el usuario: puede agregar productos, registrar un cliente, cambiar de pestaña o generar la venta en el momento que quiera. Por eso el programa no sigue una secuencia fija de instrucciones. Se queda esperando a que ocurra un evento, como un clic o la selección de un elemento, y responde ejecutando la función asociada. Este modelo es la base de casi todas las aplicaciones con interfaz gráfica, como los puntos de venta, las aplicaciones móviles y las páginas web, y por eso es importante dominarlo.

En el sistema, este modelo se implementó con tres elementos de tkinter. El primero es el ciclo principal de la ventana (`mainloop()`), que la mantiene abierta y atenta a las acciones del usuario. El segundo son las funciones que responden a cada evento, como `entrar`, `agregar`, `cancelar`, `generar_venta`, `agregar_cliente`, `guardar_articulo` y `eliminar_articulo`, que se conectaron a sus botones con la opción `command` para ejecutarse cada vez que se da clic. El tercero es el evento de selección de la tabla de artículos (`<<TreeviewSelect>>`), con el cual, al seleccionar un artículo, sus datos se cargan en los campos de captura. También se usaron ventanas emergentes (`messagebox`), que detienen el proceso hasta que el usuario responde, y un ciclo `while True` en el bloque principal que permite volver al inicio de sesión después de cerrar sesión.

Trabajar con eventos obligó a cuidar dos aspectos del diseño. Primero, como cada evento puede ocurrir en cualquier momento y con cualquier dato, cada función valida la información antes de actuar: revisa que haya un cliente y productos en la venta, que el ID y el nombre no estén vacíos, evita que el programa falle si se escribe texto donde se esperaba un número (`try / except ValueError`) y comprueba que haya existencia suficiente. Segundo, después de cada evento la interfaz debe quedar actualizada. Para eso la función `refrescar()` vuelve a llenar las listas desplegables, las tablas de clientes y artículos y el historial. Así, una venta generada en la pestaña Ventas se refleja de inmediato en las existencias de Artículos y en el Historial. Las listas `clientes`, `articulos` y `ventas` funcionan como la información compartida que conecta todos los eventos.

Los roles se implementaron con la misma idea. Al abrir la ventana principal, el sistema agrega o no las pestañas de Artículos e Historial según el rol, y con eso limita qué eventos puede generar cada usuario. El vendedor no puede administrar artículos ni consultar el historial porque esas pestañas no existen para él.

Cuestionario

1.- ¿Qué paradigmas de programación utiliza python?

Python es un lenguaje multiparadigma, es decir, permite programar con distintos estilos:

- **Imperativo y estructurado (procedural):** el programa se escribe como una secuencia de instrucciones organizada con condicionales, ciclos y funciones. Es el paradigma principal de esta práctica.
- **Orientado a objetos:** permite crear clases y objetos con atributos y métodos, herencia y encapsulamiento. En Python todo es un objeto; por ejemplo, cada botón de tkinter es un objeto de la clase Button.
- **Funcional:** las funciones son objetos que se pueden pasar como parámetros o guardar en variables, y existen lambda, map(), filter() y las comprensiones de listas. En la práctica, al escribir command=agregar se pasa una función como valor.
- **Orientado a eventos:** con librerías como tkinter, el programa espera acciones del usuario (clics, selección de elementos) y responde ejecutando funciones.

2.- ¿Cuál es la clasificación de los tipos de datos estándar en python?

Categoría	Tipos	Ejemplo	¿Mutable?
Numéricos	int, float, complex	5, 3.14, 2+3j	No
Booleano	bool	True, False	No
Texto	str	"Laptop"	No
Secuencias	list, tuple, range	[1, 2], (1, 2), range(5)	list sí; tuple y range no
Mapeos	dict	{"id": "P1"}	Sí
Conjuntos	set, frozenset	{1, 2, 3}	set sí; frozenset no
Binarios	bytes, bytearray, memoryview	b"abc"	bytearray sí; bytes no
Nulo	NoneType	None	—

Los tipos mutables pueden modificarse después de crearse (por ejemplo, agregar elementos a una lista), mientras que los inmutables no pueden cambiar; para “modificarlos” se crea un valor nuevo. En la práctica se usaron str (nombres), int y float (cantidades y precios), list y dict (clientes, artículos y ventas), tuple (fuentes y columnas) y bool.

3.- ¿Cuáles son las palabras reservadas en Python, (escribe el listado y una breve descripción)?

Las palabras reservadas son palabras que el lenguaje usa para su propia sintaxis y no pueden usarse como nombres de variables o funciones. Python 3 tiene 35

Palabra	Descripción	Palabra	Descripción
False	Valor booleano falso.	None	Representa la ausencia de valor.
True	Valor booleano verdadero.	and	Operador lógico “y”.
as	Crea un alias (import ... as, with ... as, except ... as).	assert	Verifica una condición; si es falsa lanza AssertionError.
async	Define una función asíncrona (corrutina).	await	Espera el resultado de una corrutina.
break	Termina un ciclo.	class	Define una clase.
continue	Salta a la siguiente vuelta del ciclo.	def	Define una función.
del	Elimina una variable o un elemento.	elif	Condición adicional dentro de un if.
else	Bloque alternativo de if, for, while o try.	except	Atrapa una excepción.
finally	Bloque de try que siempre se ejecuta.	for	Ciclo que recorre una secuencia.
from	Importa partes específicas de un módulo.	global	Indica que una variable es global dentro de una función.
if	Estructura condicional.	import	Importa un módulo.
in	Pertenencia (x in lista) y recorrido en for.	is	Compara si dos variables son el mismo objeto.
lambda	Crea una función anónima de una línea.	nonlocal	Usa una variable de la función que la contiene.
not	Negación lógica.	or	Operador lógico “o”.
pass	Instrucción vacía; no hace nada.	raise	Lanza una excepción.
return	Devuelve un valor y termina la función.	try	Inicia un bloque que puede generar excepciones.
while	Ciclo que se repite mientras una condición sea verdadera.	with	Administrador de contexto (por ejemplo, abrir y cerrar archivos).
yield	Devuelve valores desde un generador sin terminar la función.		

Desde Python 3.10 también existen las palabras reservadas suaves match, case y _, y desde la 3.12 type. Solo funcionan como reservadas en ciertos contextos.

4.- ¿Qué operadores de asignación se pueden utilizar en Python?

Operador	Ejemplo	Equivale a / descripción
=	x = 5	Asigna el valor 5 a x.

Operador	Ejemplo	Equivale a / descripción
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3 (división)
//=	x //= 3	x = x // 3 (división entera)
%=	x %= 3	x = x % 3 (residuo)
	x **= 3	x = x ** 3 (potencia)
&=	x &= 3	x = x & 3 (AND a nivel de bits)
|=	x |= 3	x = x | 3 (OR a nivel de bits)
^=	x ^= 3	x = x ^ 3 (XOR a nivel de bits)
>>=	x >>= 3	x = x >> 3 (desplazamiento a la derecha)
<<=	x <<= 3	x = x << 3 (desplazamiento a la izquierda)
@=	a @= b	a = a @ b (multiplicación de matrices, con librerías como NumPy)
:=	if (n := len(lista)) > 0:	Operador “morsa”: asigna un valor dentro de una expresión (Python 3.8+).

5.- ¿Cómo se realizan los comentarios de línea y multilínea en Python?

Un comentario de una línea empieza con el símbolo `#`; todo lo que sigue en ese renglón es ignorado por el intérprete. Python no tiene una sintaxis especial para comentarios multilínea: se escribe un `#` al inicio de cada renglón, o se usa una cadena entre triples comillas (`"""`) que no se asigna a ninguna variable. Cuando esa cadena va al inicio de una función, clase o módulo se llama docstring y sirve como documentación. En la práctica se usaron comentarios de una línea como `# Roles` y # Confirmar la venta.

```
# Comentario de una línea
IVA = 0.16  # Comentario al final de una instrucción


# Comentario de varias líneas
# usando un # en cada renglón


"""
Comentario de varias líneas
usando triples comillas
"""
```

6.- ¿Qué es una lista en Python y describe los métodos que utiliza para la lista?

Una lista es una colección ordenada y mutable de elementos, que pueden ser de distintos tipos y repetirse. Se escribe entre corchetes, por ejemplo colores = ["rojo", "azul"], y sus elementos se acceden por índice empezando en 0 (colores[0]). En la práctica, clientes, articulos, ventas, carrito, sesion y entradas_cli son listas.

Método	Descripción	Ejemplo
append(x)	Agrega x al final. (Usado en la práctica)	carrito.append(linea)
extend(iterable)	Agrega todos los elementos de otra colección. (Usado)	sesion.extend([usuario, rol])
insert(i, x)	Inserta x en la posición i.	a.insert(0, "x")
remove(x)	Elimina la primera aparición de x.	a.remove("x")
pop(i)	Quita y devuelve el elemento de la posición i (o el último).	carrito.pop(2)
clear()	Vacía la lista. (Usado)	carrito.clear()
index(x)	Devuelve la posición de x.	a.index("x")
count(x)	Cuenta cuántas veces aparece x.	a.count(3)
sort()	Ordena la lista.	a.sort()
reverse()	Invierte el orden.	a.reverse()
copy()	Devuelve una copia de la lista.	b = a.copy()

7.- ¿Qué es una tupla y cuáles son los métodos que se utilizan en Python para las tuplas?

Una tupla es una colección ordenada e inmutable: una vez creada no se pueden agregar, quitar ni cambiar sus elementos. Se escribe entre paréntesis, por ejemplo fuente = ("Arial", 11). Una tupla de un solo elemento necesita una coma: (5,). En la práctica se usaron tuplas para las fuentes (font=("Courier", 9)), para las columnas de las tablas (columns=("id", "nombre", "telefono")), para devolver varios valores en mostrar_ticket() (return subtotal, iva) y al leer tabla_art.selection() y tabla_art.item(sel[0], "values"), que regresan tuplas. Como son inmutables, solo tienen dos métodos:

Método	Descripción	Ejemplo
count(x)	Cuenta cuántas veces aparece x.	(1, 2, 1).count(1) → 2
index(x)	Devuelve la posición de la primera aparición de x.	("a", "b").index("b") → 1

Además, las tuplas admiten operaciones comunes de secuencias como len(), indexado, rebanadas, in y desempaquetado (a, b = (1, 2)).

8.- ¿Qué son los conjuntos en Python y cuales son los métodos que se utilizan para los conjuntos?

Un conjunto (set) es una colección no ordenada de elementos únicos: no permite duplicados ni acceso por índice. Se escribe entre llaves, por ejemplo ids = {"P1", "P2"}; un conjunto vacío se crea con set(), porque {} crea un diccionario vacío. Es útil para eliminar duplicados y para operaciones de teoría de conjuntos. En esta práctica no se usaron, pero podrían servir, por ejemplo, para detectar IDs repetidos al registrar clientes o artículos.

Método	Descripción	Ejemplo
add(x)	Agrega un elemento.	s.add("P7")
update(iterable)	Agrega varios elementos.	s.update([1, 2])
remove(x)	Elimina x; da error si no existe.	s.remove(1)
discard(x)	Elimina x sin dar error si no existe.	s.discard(1)
pop()	Quita y devuelve un elemento cualquiera.	s.pop()
clear()	Vacía el conjunto.	s.clear()
union(otro)	Elementos de ambos conjuntos (|).	a.union(b)
intersection(otro)	Elementos en común (&).	a.intersection(b)
difference(otro)	Elementos de a que no están en b (-).	a.difference(b)
symmetric_difference(otro)	Elementos que están en uno u otro, pero no en ambos (^).	a.symmetric_difference(b)
issubset(otro)	¿Todos sus elementos están en el otro?	a.issubset(b)
issuperset(otro)	¿Contiene todos los elementos del otro?	a.issuperset(b)
isdisjoint(otro)	¿No tienen elementos en común?	a.isdisjoint(b)
copy()	Devuelve una copia.	b = a.copy()

9.- ¿Qué estructuras de control utilizaste en el desarrollo de la practica?

Estructura	Uso en la práctica
if / else	Validar usuario y contraseña, campos vacíos, cantidades y existencias; aplicar los permisos según el rol (if rol == "Administrador").
if not	Detectar un carrito vacío o la falta de cliente (if not carrito or not cb_cliente.get()).
for	Recorrer el carrito, los clientes, los artículos y las ventas para mostrarlos, sumar importes y descontar existencias.

Estructura	Uso en la práctica
while True / break	Ciclo del bloque principal que muestra el inicio de sesión y abre la tienda; termina con break si se cierra el login sin iniciar sesión.
try / except	Atrapar el error ValueError al convertir a número lo que escribe el usuario.
return	Salir de una función antes de tiempo cuando un dato no es válido, o devolver un resultado.
Comprensiones	Construir listas y buscar elementos en una sola línea, por ejemplo con sum(... for ...) y next(... for ... if ...).

Ejemplo de for con if al descontar las existencias de los productos vendidos:

```
for p in carrito:
    p["art"]["cantidad"] -= p["cantidad"]
```

Ejemplo de try / except al leer la cantidad:

```
try:
    cant = int(e_cantidad.get())
except ValueError:
    cant = 0
```

Ejemplo del ciclo while del bloque principal:

```
while True:
    sesion = login()
    if not sesion:
        break
    tienda(*sesion)
```

10.- Si utilizaste alguna de las funciones de Python, describe el procedimiento realizado para utilizarla

En la práctica se crearon funciones propias y se usaron funciones integradas de Python. El procedimiento para utilizarlas fue el siguiente:

1. Definir la función con def, su nombre, sus parámetros entre paréntesis y dos puntos. El cuerpo se escribe indentado, por ejemplo def tienda(usuario, rol):. También se puede dar un valor predeterminado a un parámetro, como en mostrar_ticket(encabezado="VENTA EN CURSO").
2. Devolver resultados con return. Se pueden devolver varios valores separados por comas, que Python agrupa en una tupla, como en return subtotal, iva.
3. Llamar a la función escribiendo su nombre con paréntesis y los argumentos: subtotal, iva = mostrar_ticket(). Con tienda(*sesion) los elementos de la lista se envían como argumentos.

4. Conectarla a un botón pasando la función sin paréntesis en la opción command, por ejemplo command=agregar. Así tkinter la ejecuta cada vez que se da clic y no solo una vez al crear el botón.
5. Conectarla a un evento con bind(). Por ejemplo, tabla_art.bind("<<TreeviewSelect>>", seleccionar_art); en este caso la función recibe un parámetro event que tkinter le envía automáticamente.
6. Definir funciones dentro de otras funciones. entrar() está dentro de login() y agregar(), cancelar() y generar_venta() están dentro de tienda(), por lo que pueden usar las variables de la función que las contiene (carrito, ticket, cb_cliente, etc.).

Funciones integradas y métodos de Python utilizados:

Función o método	**Para qué se usó**
int() / float()	Convertir a número lo que el usuario escribe en las cajas de texto (cantidad y precio).
len()	Obtener el folio de la venta (len(ventas) + 1).
sum()	Sumar los importes del carrito y las cantidades ya agregadas de un producto.
next()	Buscar un artículo por su ID; regresa None si no existe.
zip() / enumerate()	Recorrer a la vez los campos y sus valores, y crear los campos de captura con su posición.
.strip()	Quitar espacios al inicio y al final de lo que escribe el usuario.
.join()	Unir las líneas del ticket en un solo texto.
.get()	Leer el contenido de una caja de texto o buscar un usuario en el diccionario usuarios.
.append(), .extend(), .clear(), .copy()	Agregar, extender, vaciar y copiar elementos de las listas y los diccionarios.
.update()	Actualizar los datos de un artículo (diccionario) al editarlo.
datetime.now().strftime()	Obtener la fecha y hora actual con formato día/mes/año hora:minuto para cada venta.
f-strings	Dar formato y alinear los números y el texto del ticket, por ejemplo {total:>11.2f}.

Referencias Bibliográficas

Covatec (2018), Programación en Python-nivel básico, disponible en: https://entrenamiento-python-basico.readthedocs.io/es/latest/index.html

López Ostenero Fernando, García Serrano Ana María (2014), Teoría de los Lenguajes de Programación, Editorial Universitaria Ramón Areces, ISBN: 9788499611396.

Python Software Foundation (s. f.), tkinter — Interfaz de Python para Tcl/Tk, disponible en: https://docs.python.org/es/3/library/tkinter.html

Píldoras Informáticas (2018a). Curso Python. Interfaces gráficas I. Vídeo 42 [Video]. YouTube: https://youtu.be/hTUJC8HsC2I?si=Za2gmwxyNV1WuZvF

Píldoras Informáticas (2018b). Curso Python. Interfaces gráficas II. Vídeo 43 [Video]. YouTube: https://youtu.be/M80CzDC1Crc?si=5S5VGBKvKVV-8nHz

PíldorasInformáticas(2018c) CursoPython InterfacesgráficasIII Vídeo44[Video] YouTube:https://youtube/Nf4gvf